

IMCS3010U Final Project Report: Textual/Binary Classification of Mathematical Vs. Non-Mathematical Textual Content

Tobenna Chizurumoke Nnaobi

https://github.com/Toby-pixelRev/IMCS3010UProject_TextBinary_Classification_Math_Vs_NonMath

Tuesday, December 2nd, 2025

Contents

1	Introduction	2
2	The Problem Statement	2
3	Methodologies Overview	2
3.1	Data Preparation	2
3.2	Human Interpretation Testing Phases	5
3.2.1	Testing Phase: Concordance	5
3.2.2	Testing Phase: Cosine Similarity	5
3.3	Computation Interpretation Testing Phase	5
3.3.1	Testing Phase: TensorFlow Prediction Recurrent Neural Network (TFP-RNN)	5
4	Mathematical & Computational Backgrounds	6
4.1	Defining Natural Language Processing (NLP)	6
4.2	Defining Concordance	6
4.3	Defining Cosine Similarity	7
4.4	Defining Recurrent Neural Networks (RNNs)	7
5	The Resultant Data Showcase & Post-Results Discussion	9
5.1	Human Interpretation Testing Results	9
5.1.1	Testing Phase: Concordance	9
5.1.2	Testing Results: Cosine Similarity	10
5.2	Computational Interpretation Testing Results	11
5.2.1	Testing Results: TensorFlow Prediction Recurrent Neural Network (TFP-RNN)	11
6	Conclusion	14
7	The Future of This Project	14
8	Citations	15

Abstract

In this final project, critical aspects of the state of the finalized project will be discussed, such as its problem statement, an overview of the methodologies used, showcasing the resultant data, analyzing and thoroughly discussing the resultant data, the various challenges that arose during the project, discussing the underlying conclusions for this project, and the potential expansion of this project.

1 Introduction

One of the most fascinating aspects of human connection and society at large is the use of language as a form of communication. Throughout the many languages that span various cultural, regional, and global communities, including English, Igbo, Japanese, French, Bengali, and Hindi, and their differing dialects by region, there is one specific language that transcends cultural borders. It is no surprise that mathematical linguistics is considered a universal language that transcends global borders. However, one of the most fascinating aspects of this language is the existence of shared terms with other standardized languages that contain completely different definitions and interpretations depending on how it is used and the verbal and textual context surrounding its usage.

2 The Problem Statement

As mentioned above, one of the most fascinating aspects of languages is the existence of shared terms with other standardized languages that contain completely different definitions and interpretations depending on how it is used and the verbal and textual context surrounding its usage.

In particular, mathematical language shares many terms with standardized English, one of the languages of Latin origin, which contain definitions and contextual interpretations that differ from their standardized English linguistic definitions and interpretations.

This discovery raised an important question: “Is a computer able to computationally differentiate between a mathematical and non-mathematical piece of text containing shared terms, and if so, is there a variety of analytical steps and methodologies to showcase that this is a possibility?” This question resulted in the primary research topic for this project, which is the Textual/Binary Classification of Math vs. Non-Math Textual Content.

3 Methodologies Overview

3.1 Data Preparation

Data preparation involves the qualitative data collection process by collecting various free mathematical resources with an abundance of textual content using the various libraries and functionalities for text preprocessing, such as filtering out stop words, stemming, and lemmatizing words.

Due to the lack of usable datasets available for a project like this, a custom qualitative dataset was built and personally constructed for this specific project. This custom data set will be constructed using the Pandas DataFrame functionality.

The mathematical text content are then stored in LaTeX files. For this, File I/O will be utilized to read the LaTeX file and store its code in a data variable, and the `latex_to_text()` function found

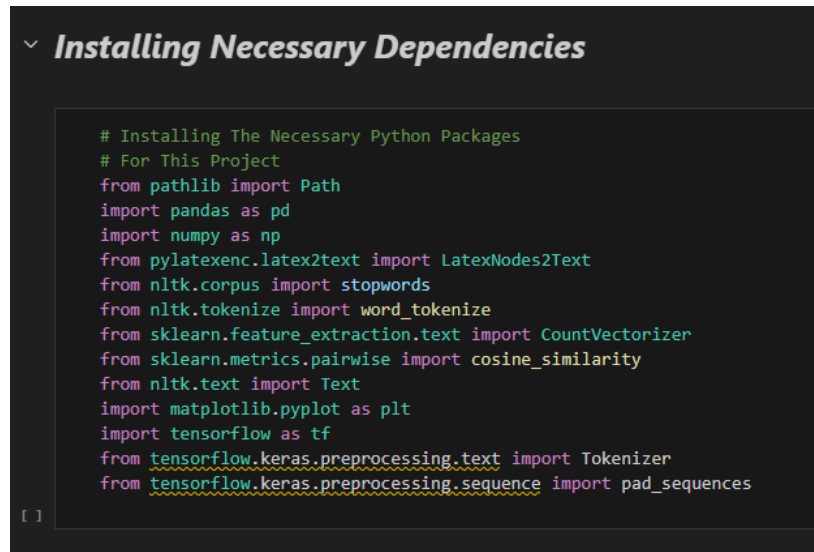
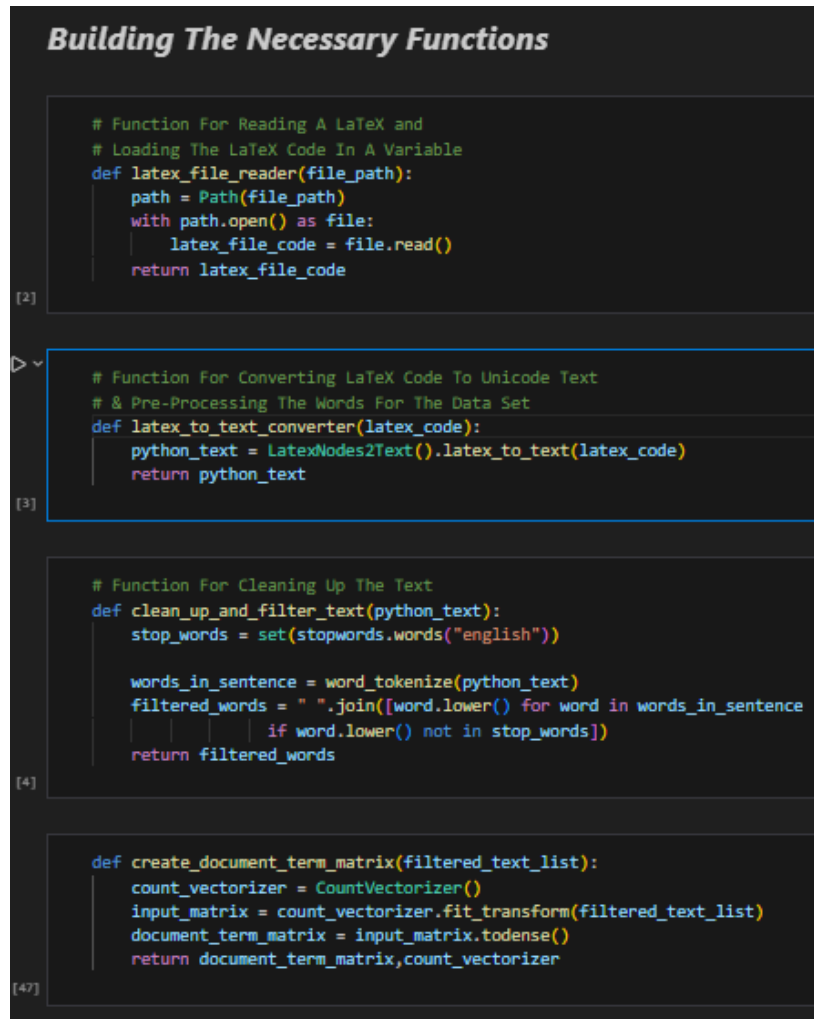


Figure 1: Installing The Necessary Dependencies For This Project.

in the pylatexenc library will also be utilized for the purpose of converting LaTeX code to Python Unicode text for easier preprocessing.

Once all textual content has been converted into Python Unicode text, the text will be pre-processed utilizing the stopwords and word.tokenize() functionalities in the NLTK libraries, which will remove stop words in a piece of text and tokenize each word in a sentence for easier interpretation. The pre-processed texts will then be stored in the custom dataset as a list, in which each text will have a numeric value of either 1 or 0 assigned to it stored in a separate list in the dataset (1 means that the topic of the text is Math and 0 means that the topic of the text is Non-Math).



```
Building The Necessary Functions

# Function For Reading A LaTeX and
# Loading The LaTeX Code In A Variable
def latex_file_reader(file_path):
    path = Path(file_path)
    with path.open() as file:
        latex_file_code = file.read()
    return latex_file_code
[2]

# Function For Converting LaTeX Code To Unicode Text
# & Pre-Processing The Words For The Data Set
def latex_to_text_converter(latex_code):
    python_text = LatexNodes2Text().latex_to_text(latex_code)
    return python_text
[3]

# Function For Cleaning Up The Text
def clean_up_and_filter_text(python_text):
    stop_words = set(stopwords.words("english"))

    words_in_sentence = word_tokenize(python_text)
    filtered_words = " ".join([word.lower() for word in words_in_sentence
                               if word.lower() not in stop_words])
    return filtered_words
[4]

def create_document_term_matrix(filtered_text_list):
    count_vectorizer = CountVectorizer()
    input_matrix = count_vectorizer.fit_transform(filtered_text_list)
    document_term_matrix = input_matrix.todense()
    return document_term_matrix, count_vectorizer
[47]
```

Figure 2: Building The Necessary Functions For This Project.

The functionality of Sci-kit Learn, `CountVectorizer()`, will be used to construct a Document-Term Matrix (DTM) with each entry representing the number of times a word appears, or in this case, a “term” appears in a text. This matrix will be useful when conducting Cosine Similarity testing. NLTK’s functionality, `Text()`, will be used to construct an NLTK Text object out of non-preprocessed textual content. This will be useful for conducting Concordance testing, as this will not function without the non-preprocessed text being converted into an NLTK Text object.

3.2 Human Interpretation Testing Phases

3.2.1 Testing Phase: Concordance

This testing phase will involve using the qualitative data from the NLTK Text object and viewing the contextual use cases of a specific word within the data to gain a better understanding on the subject matter, or “topic”, of a piece of textual content.

Once the NLTK Text Object has been initialized, the NLTK functionality, `concordance()` will be used to check the specific use cases of a word in a piece of text. This method will correctly identify the subject matter of piece of text depending on how the word is being used in a specific sentence.

3.2.2 Testing Phase: Cosine Similarity

This testing phase will involve using the qualitative data from the DataFrame created to measure the similarity of a piece of text in relation to all other textual content in the dataset.

Once the text in the data set has been preprocessed and the Document Term Matrix (DTM) has been created, the next step is using the Pandas DataFrame functionality and Sci-kit Learn’s `CountVectorizer()` functionality to create a data set based on the DTM that creates a detailed list of all texts in the dataset. The rows represent all text content in the dataset, and the columns represent key words obtained from all text content. For every row, each column will assign a value of 0 or 1 depending on whether a keyword appears in a piece of text (0 means it does not exist and 1 means it exists).

The input values are as follows: The DTM will be used as the input matrix as a base, the columns will use the `CountVectorizer()`’s sub-functionality `get_feature_names_out()` to obtain all keywords every text, using them as column values. Each row, or “index in the case, will label each text in the order of the assigned placement in the initialized text array.

After everything has been initialized, Sci-kit Learn’s `cosine_similarity()` functionality will be used to create a cosine similarity matrix where every column represents a piece of text, with each row representing the cosine similarity vector direction between -1 and 1 in relation to every other text in the data set. (1 represents complete similarity, 0 represents orthogonality and no similarity in direction, and -1 represents complete dissimilarity).

3.3 Computation Interpretation Testing Phase

3.3.1 Testing Phase: TensorFlow Prediction Recurrent Neural Network (TFP-RNN)

This testing phase will involve using a Recurrent Neural Network (RNN) built with the TensorFlow library to have the computer make a prediction on whether a piece of textual content is either Math or Non-Math based on the qualitative data inserted into the RNN Model after the completion of its training phase. After the DataFrame has been initialized, the RNN model will need to be built using several functionalities within the TensorFlow library. Due to the complexity of building the RNN model, however, I will only mention a few functionalities of the RNN model and what they are responsible for:

- The `Sequential()` function will be used to construct the RNN model with the `summary()` functionality printing out critical information such as Layer information, the Output shape of the results, and the number of Params being used.
- The `compile()` functionality will be used to compile the RNN model post-construction, and the `fit()` functionality will train the model using Binary Classification by using a converted version

of textual sample data that are a row of numbers called a padded sequence so that the computer is able to read the textual data in a computational language.

After training is complete, matplotlib's pyplot functionality will be used to convert the training data into a graphical representation.

A predictions array for the testing phase will be constructed using TensorFlow's predict() functionality, which contains all the predicted percentage values during the testing phase after inputting the testing data.

The last step is constructing a simple for loop to output the predicted values of the testing results.

4 Mathematical & Computational Backgrounds

4.1 Defining Natural Language Processing (NLP)

Natural Language Processing (NLP) is a subfield of computer science and artificial intelligence that utilizes machine learning to enable computers to understand and communicate with human language.[10] NLP works by combining various computational techniques to analyze, interpret, and generate human language in a manner that machines can process. Some of the dependencies utilized for NLP models include, but are not limited to:

- **NLTK:** Also known as Natural Language Toolkit, it is considered a leading platform for building Python programs to work with human language data. It provides easy-to-use corpora and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning[6].
- **PyLaTeXEnc:** A Python resource that can convert Unicode text to LaTeX code and vice versa. It also provides functions for parsing LaTeX code and converting it into plain text[7].
- **Scikit-Learn:** A Python module that is utilized for machine learning purposes and contains tools for predictive data analysis. It is built on NumPy, SciPy, and matplotlib[9].
- **Matplotlib:** A powerful Python dependency that used to display graphical representations of pieces of critical data[3].
- **TensorFlow:** A free open-source dependency in Python developed by the now defunct Google Brain[1]. It is usually used for computational development specifically within the realm of machine-learning and artificial intelligence (AI)[11].

4.2 Defining Concordance

In the world of linguistics, a concordance is a listing of each word, or "term", in a piece of textual content, presented with the words surrounding it[16]. Additionally, the dependency that will be utilized for this computational background include:

- **nlk.text.concordance:** In data visualization, it is a dependency derived from the nltk.text module, used to computationally visualize the contextual use cases of specified word or "term"[5].

4.3 Defining Cosine Similarity

In data analysis, Cosine Similarity is a widely used similarity metric that determines how similar two data points are based on the direction the vectors are pointing rather than their length or size. It is especially effective in high-dimensional spaces where traditional distance-based metrics can struggle:

$$\text{similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2 \sum_{i=1}^n B_i^2}}.$$

Figure 3: Cosine Similarity Equation

The process of computing the cosine similarities of two data points requires measuring the cosine of the angle (θ) between two non-zero vectors, defined in an inner-product space[2]. This measurement produces a cosine similarity score ranging from -1 to 1.

- A cosine similarity score of 1 indicates that the vectors point in the same direction. (Topical Similarity)
- A cosine similarity score of 0 indicates that the vectors are orthogonal, meaning they have no directional similarity. (No Topical Correlation)
- A cosine similarity score of -1 indicates that the vectors are pointing in opposite directions. (Topical Dissimilarity)

This mathematical and computational background will be utilized to determine the similarity of non-mathematical and mathematical texts in a data set.

Additionally, some of the dependencies that will be utilized for this mathematical background uses:

- `sklearn.metrics.pairwise.cosine_similarity`: A type of metric utilized to compute the cosine similarity between two samples in X and Y[8].

4.4 Defining Recurrent Neural Networks (RNNs)

A Recurrent Neural Network is a type of computational neural network that is designed to process sequential data such as text and speech. Unlike feedforward neural networks, RNNs utilize recurrent connections, where the output of a neuron is fed back as input to the network, enabling them to capture patterns within sequences [15]. Additionally, some of the dependencies that will be utilized for this mathematical background uses:

- `tf.keras.Sequential`: A dependency that is used to group a linear stack of layers into an RNN Model[14].
- `tf.keras.layers`: A dependency that is used to provide a list of fundamental building components of a recurrent neural network[12].

- `tf.keras.preprocessing.text.Tokenizer`: A dependency used to initialize a tokenizer that will transform a textual dataset into a padded sequence of numbers[13].
- `matplotlib.pyplot`: A state-based interface to `matplotlib`. It provides an implicit, MATLAB-like, way of plotting graphical representations. `pyplot` is mainly intended for interactive plots and simple cases of programmatic plot generation[4].

5 The Resultant Data Showcase & Post-Results Discussion

5.1 Human Interpretation Testing Results

5.1.1 Testing Phase: Concordance

```

# Convert the filtered text in the dataset into a
# NLTK Text Object for "Sequence".
filtered_sequences_math_text_obj1 = Text(word_tokenize(filtered_sequences_data["text"][0]))
filtered_sequences_math_text_obj2 = Text(word_tokenize(filtered_sequences_data["text"][1]))
filtered_sequences_text_obj3 = Text(word_tokenize(filtered_sequences_data["text"][2]))
filtered_sequences_non_math_text_obj = Text(word_tokenize(filtered_sequences_data["text"][3]))

# The display the contextual use cases
# for the word "Sequence" for each piece
# of text.
filtered_sequences_math_text_obj1.concordance("sequence")
print("\n")

filtered_sequences_math_text_obj2.concordance("sequence")
print("\n")

filtered_sequences_text_obj3.concordance("sequence")
print("\n")

filtered_sequences_non_math_text_obj.concordance("sequence")
print("\n")

```

[44]

```

...
Displaying 5 of 5 matches:
rs 2 , 4 , 6 , 8 , 10 , ... , form sequence . order important ; first number 2
tural seek formula describes given sequence , often done . instance , sequence
sequence , often done . instance , sequence could bedescribed function ( n ) =
1,2,3,4 , .. , k . find 10^th term sequence , would compute ( 10 ) . leads us
s us following , formal definition sequence .

Displaying 1 of 1 matches:
let { a_n } sequence let L real number . given  $\epsilon > 0$  ,

Displaying 2 of 2 matches:
nition states , informally , limit sequence L , go far enough along sequence ,
t sequence L , go far enough along sequence , subsequent terms really close L

Displaying 1 of 1 matches:
uld need stay informed . however , sequence events occurred student society one

```

Figure 4: Takes the word "Sequence", promptly showcasing how many times it has been used in this data set and shows the various contextual use cases.

When I was examining the results of the Concordance testing phase, it is evident that this functionality actively showcases the contextual use cases for the shared terms of sequence and series. In the non-mathematical text example for the word "series", the word series is used when talking about the subject matter of a person's list of favourite television shows, while in the mathematical text example for the word "sequence", the word is clearly used in a mathematical context. Therefore, this testing phase was relatively successful.

5.1.2 Testing Results: Cosine Similarity

```
# Create The Document Term Matrix For
# The Filtered "Sequence" Text
document_term_matrix_sequence,count_vectorizer_sequence = create_document_term_matrix(filtered_text_sequences)
document_term_matrix_sequence

[89]
... matrix([[3, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0,
            1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0,
            0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0,
            0, 0, 0, 0, 1, 1, 1, 5, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0,
            0],
            [0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0,
            0, 0, 0, 0, 0, 2, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
            1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            3],
            [0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
            0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
            0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 1, 0, 0, 0, 0, 0, 2, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
            0],
            [0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1,
            0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 2, 1, 0, 0, 0,
            1, 1, 0, 0, 1, 0, 0, 0, 1, 2, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1,
            0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 2, 0, 0, 0, 0, 0, 2, 1,
            0]])
```

Figure 5: Created The Document-Term Matrix (DTM) For The Term "Sequence".

When I was examining the results of the Cosine Similarity testing phase, something interesting was discovered in the resultant cosine similarity matrix.

When a person is interpreting the results during this phase, every column represents all textual content present in the dataset with every row representing how the present text is related to the other texts in the dataset. What was incredibly noticeable is the results of the cosine similarity matrix of the Series dataset.

When observing the results of the cosine similarity matrix of the Series dataset, one of the inter-

```
# Print The Cosine Similarity Matrix For "Sequence"
print(cosine_similarity(cosine_sim_text_sequence_data,cosine_sim_text_sequence_data))

[58]
... [[1.         0.16854019 0.3310611  0.12562244]
      [0.16854019 1.         0.15      0.02981424]
      [0.3310611  0.15      1.         0.1118034 ]
      [0.12562244 0.02981424 0.1118034  1.         ]]
```

Figure 6: Uses the DTM to create a cosine similarity matrix for the term "Sequence".

esting aspects that was discovered was the matrix showcasing that two differing mathematical texts had a cosine similarity vector value of 1 while the non-mathematical text had cosine similarity vector values ranging from 0.102 to 0.162 compared to every mathematical text, meaning that the matrix recognized the non-mathematical but found complete similarity of the two math texts.

The matrix recognized that the non-mathematical text in relation to other text, besides itself, received a vector value close to 0. This indicates that the computer understood one of the texts is non-math despite sharing the term "Series". Therefore, this testing phase was relatively successful!

5.2 Computational Interpretation Testing Results

5.2.1 Testing Results: TensorFlow Prediction Recurrent Neural Network (TFP-RNN)

```
# Build The RNN Model
sequence_model = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim=5000,
        output_dim=64,
        input_length=50),
    tf.keras.layers.SimpleRNN(64, return_sequences=False),
    tf.keras.layers.Dense(1, activation='sigmoid'),
])

sequence_model.summary()
```

[23]

```
...
Model: "sequential_2"
...
```

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
simple_rnn_2 (SimpleRNN)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

```
...
Total params: 0 (0.00 B)
...
Trainable params: 0 (0.00 B)
...
Non-trainable params: 0 (0.00 B)
```

Figure 7: Building The RNN Model For The Term "Sequence".

When I was examining results of the TF-PRNN testing phase, one of the more interesting aspects of the training phase was that, despite the RNN Model correctly identifying the Math and Non-Math textual content for the shared terms, the stark difference between the training results of the Sequence RNN Model and the Series Model.

```

# Compile The RNN Model
sequence_model.compile(
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(),
    metrics=["accuracy"])

```

Figure 8: Compiling The RNN Model For The Term "Sequence".

```

# Training The Model
sequence_words_tokenizer = Tokenizer(num_words=5000)
sequence_words_tokenizer.fit_on_texts([text_sequences_df["text"][1],
                                       text_sequences_df["text"][2],
                                       text_sequences_df["text"][3]])

# Create Training Content
train_sequences_of_sequences = sequence_words_tokenizer.texts_to_sequences([text_sequences_df["text"][1],
                                                                              text_sequences_df["text"][2],
                                                                              text_sequences_df["text"][3]])
sequences_train = pad_sequences(train_sequences_of_sequences,
                                maxlen=50,
                                padding='post')
sequences_train_labels = np.array([text_sequences_df["label"][1],
                                   text_sequences_df["label"][2],
                                   text_sequences_df["label"][3]])

# Create Testing Content
test_sequences_of_sequences = sequence_words_tokenizer.texts_to_sequences([text_sequences_df["text"][0]])
sequences_test = pad_sequences(test_sequences_of_sequences,
                               maxlen=50,
                               padding='post')
sequences_test_labels = np.array([text_sequences_df["label"][0]])

sequence_history = sequence_model.fit(sequences_train,
                                       sequences_train_labels,
                                       epochs=10,
                                       batch_size=2,
                                       validation_data = (sequences_test, sequences_test_labels))

```

```

Epoch 1/10 2/2 3s 483ms/step - accuracy: 0.3333 - loss: 0.7146 - val_accuracy: 1.0000 - val_loss: 0.5431
Epoch 2/10 2/2 0s 66ms/step - accuracy: 1.0000 - loss: 0.4212 - val_accuracy: 1.0000 - val_loss: 0.5021
Epoch 3/10 2/2 0s 78ms/step - accuracy: 1.0000 - loss: 0.2384 - val_accuracy: 1.0000 - val_loss: 0.4445
Epoch 4/10 2/2 0s 80ms/step - accuracy: 1.0000 - loss: 0.1524 - val_accuracy: 1.0000 - val_loss: 0.3570
Epoch 5/10 2/2 0s 77ms/step - accuracy: 1.0000 - loss: 0.1056 - val_accuracy: 1.0000 - val_loss: 0.2877
Epoch 6/10 2/2 0s 78ms/step - accuracy: 1.0000 - loss: 0.0777 - val_accuracy: 1.0000 - val_loss: 0.2528
Epoch 7/10 2/2 0s 106ms/step - accuracy: 1.0000 - loss: 0.0629 - val_accuracy: 1.0000 - val_loss: 0.2503
Epoch 8/10 2/2 0s 85ms/step - accuracy: 1.0000 - loss: 0.0511 - val_accuracy: 1.0000 - val_loss: 0.2543
Epoch 9/10 2/2 0s 78ms/step - accuracy: 1.0000 - loss: 0.0402 - val_accuracy: 1.0000 - val_loss: 0.2337
Epoch 10/10 2/2 0s 71ms/step - accuracy: 1.0000 - loss: 0.0318 - val_accuracy: 1.0000 - val_loss: 0.1988

```

Figure 9: Training The RNN Model For The Term "Sequence".

What really caught my attention was the fact that the validation accuracy remained consistent throughout in both graphs. However, the loss values had a steep decline in the Series RNN training results, but then the loss values were almost oscillating in the Sequence RNN training results.

However, this model is not perfect. Because of, for lack of a better term, the *fragile* nature of TensorFlow RNN model, there were numerous times where the prediction results after the data collection phase of the training had issues of inconsistent outputted data becoming a regular occurrence.

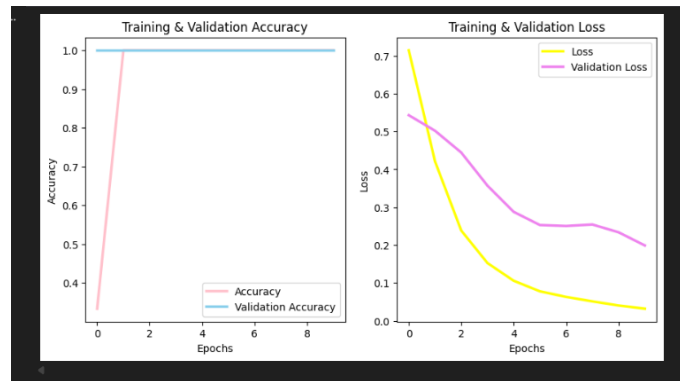


Figure 10: Plotting The Training Results.

There have even been cases where specific forms data could not be accessed due to errors in data construction in the RNN model construction process (e.g. creating the `Tokenizer()` so that the textual data can be converted into a padded sequence of numbers for the computer to process in the model).

Testing Phase (RNN)

```
# Testing The RNN Model
testing_sequences_sequences = sequence_words_tokenizer.texts_to_sequences(
    text_sequences_data["text"]
)

testing_sequences = pad_sequences(testing_sequences_sequences,
                                  maxlen=50,
                                  padding='post')

predictions = sequence_model.predict(testing_sequences)
for i in range(len(predictions)):
    if (predictions[i] > 0.65):
        print("This sentence is math related!")
    else:
        print("This sentence is not math related!")
```

1/1 ————— 0s 167ms/step
This sentence is math related!
This sentence is math related!
This sentence is math related!
This sentence is not math related!

Figure 11: Testing The RNN Model For The Term "Sequence".

6 Conclusion

After looking through the resultant data of all forms of testing, the results showcase that while a computer can computationally differentiate between Math and Non-Math textual content regardless of shared terms, the overall effectiveness of its computational understanding and interpretation of text with shared terms displays varying degrees depending on the model that was used.

Additionally, while the desired goals for this project were satisfied or completed, there are few improvements that this project could implement that would have produced more concrete results for this project. For example, a more effective data set with an equal amount and non-mathematical and mathematical textual content would have prevented the RNN model from producing radically inconsistent training data results, allowing for more cleaner graphical representations and easier-to-read data.

7 The Future of This Project

While working on this project, a realization was reached on how this topic can truly shine. However, this project expansion will have to be saved for the upcoming Integrated Mathematics & Computer Science Thesis project due to the sheer scope of the expansion. While the true details of this project expansion will be kept under wraps until the near future, I have decided that a potential expansion of this project will surround the topic of: ***"Textual & Binary Classification: Computationally Differentiating Between Different Emotions & Tones Across Different Languages via Texting"***.

8 Citations

References

- [1] GeeksForGeeks. *Introduction To TensorFlow*. URL: <https://www.geeksforgeeks.org/python/introduction-to-tensorflow/> (visited on 12/01/2025).
- [2] Tom Krantz and Alexandra Jonker. *What is cosine similarity?* URL: <https://www.ibm.com/think/topics/cosine-similarity> (visited on 10/20/2025).
- [3] Matplotlib. *Matplotlib: Visualization with Python*. URL: <https://matplotlib.org/> (visited on 12/01/2025).
- [4] Matplotlib. *matplotlib.pyplot*. URL: https://matplotlib.org/stable/api/ pyplot_summary.html (visited on 10/20/2025).
- [5] NLTK. *concordance(word, width=79, lines=25)*. URL: <https://www.nltk.org/api/nltk.text.html#nltk.text.Text.concordance> (visited on 12/01/2025).
- [6] NLTK. *Natural Language Toolkit*. URL: <https://www.nltk.org/> (visited on 10/20/2025).
- [7] PyPi. *pylatexenc 2.10*. URL: <https://pypi.org/project/pylatexenc/> (visited on 10/20/2025).
- [8] Scikit-Learn. *cosine_similarity*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html (visited on 10/20/2025).
- [9] Scikit-Learn. *scikit-learn*. URL: <https://scikit-learn.org/stable/> (visited on 10/20/2025).
- [10] Cole Stryker and Jim Holdsworth. *What is NLP?* URL: <https://www.ibm.com/think/topics/natural-language-processing/> (visited on 10/20/2025).
- [11] TensorFlow. *TensorFlow*. URL: <https://www.tensorflow.org/> (visited on 12/01/2025).
- [12] TensorFlow. *tf.keras.layers*. URL: https://www.tensorflow.org/api_docs/python/tf/keras/layers (visited on 12/01/2025).
- [13] TensorFlow. *tf.keras.preprocessing.text.Tokenizer*. URL: https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/text/Tokenizer (visited on 12/01/2025).
- [14] TensorFlow. *tf.keras.Sequential*. URL: https://www.tensorflow.org/api_docs/python/tf/keras/Sequential (visited on 12/01/2025).
- [15] Wikipedia. *Recurrent neural network*. URL: https://en.wikipedia.org/wiki/Recurrent_neural_network (visited on 12/01/2025).
- [16] Martin Wynne. *Searching Concordancing*. URL: <https://www.pala.ac.uk/searching-and-concordancing.html> (visited on 12/01/2025).